

Passive Pre-Association Wi-Fi Access-Point Selection

U.K.Arvinandan
EP21B006

Abstract— A Wi-Fi client in overlapping coverage must select an access point (AP) before it can directly measure the service that each AP would provide. Signal strength describes link quality, but not contention or load, and may therefore favor a nearby but congested AP. This project investigates whether a client can make a better decision using only passive observations collected before association without assistance from the network.

We build a controlled ns-3 simulator of multi-AP 802.11n deployments in which each deployment is replayed once per AP, with everything before association held fixed, so that one passive observation is paired with the throughput every AP would have delivered. The observation is projected onto what a single radio cycling through the channels would hear. A hierarchical transformer predicts each AP's throughput from this observation, with either a small backbone trained from scratch or, following NetLLM, a LoRA-adapted Llama-3.2-1B. Over 1,000 simulated deployments, the small model selects the best AP in 88.8% of scans with a mean regret of 0.42 Mbit/s, against 62.1% and 5.26 Mbit/s for strongest-signal selection. Gradient boosting on whole-window summaries comes within error margins, and the 1B-parameter LLM backbone performs comparably to the bespoke model with 0.35 million trainable parameters.

I. PROBLEM FORMULATION

Throughput depends on channel contention, competing stations, transmission rates, retries, and the airtime consumed by neighboring basic service sets. Received signal strength reflects such effects poorly if at all. Those effects do, however, leave traces a client can hear without transmitting: the frames themselves, and the time the medium spends busy. This project asks whether such traces are enough: *can a client choose the AP that will provide the greatest throughput using only what its radio can passively observe before association?*

Existing load-aware methods [1], [2] make their load estimate after association, from AP-provided reports, or from a network controller. A client making its first choice has none of these: only beacons and some data frames, in which AP distance and offered traffic appear only through their effects.

Let $\mathcal{A} = \{a_1, \dots, a_{N_{AP}}\}$ be the APs in one deployment and let O be the candidate station's passive observation before it joins any AP. If y_i is the throughput obtained after joining a_i , the desired decision is

$$a^* = \arg \max_{a_i \in \mathcal{A}} y_i, \quad (1)$$

using a decision function whose inputs are derived only from O .

We tackle this problem by predicting the value of y_i for each a_i .

It is worth noting that models that simply output a ranking of the available choices are viable as well. In the simulated data we ensure that we have true counterfactuals by deliberately verifying O to be identical across all N_{AP} runs of a given scenario. In a live network however, only one y_i is ever measurable: the station joins a single AP and observes that AP's throughput, while the values of y_j for every AP it did not join are not easily recoverable. For broader applicability, and adaptation to reality, choice models are not investigated extensively here.

II. SIMULATOR DESIGN AND ASSUMPTIONS

A. Simulation Environment

The simulator is built on ns-3.45 and its Wi-Fi models [8]. The deployment is static and single-floor with no walls, the environment TGax assumes for its indoor small-BSS scenario [5]. All stations are immobile and remain at their seeded positions. Radios are homogeneous IEEE 802.11n devices at 5 GHz on 20 MHz channels with Minstrel-HT rate control [8].

Every radio is an ns-3 `SpectrumWifiPhy` attached to one shared `MultiModelSpectrumChannel`, on which a log-distance loss (exponent 3.0, with 46.6777 dB of reference loss at 1 m) is followed by Nakagami fading.

`SpectrumWifiPhy` is chosen over the simpler `YansWifiPhy` as the Yans model simulates exactly zero cross channel interference by treating each channel as an isolated layer. If `YansWifiPhy` is used, the channel occupancy given by CCA measurements becomes a powerful signal for models in a way that does not reflect reality.

B. Count, Topology and Channel occupancy of APs

Each scenario first draws 2, 3, 4, 6, or 8 APs. They sit on a triangular lattice 20 m apart (Fig. 1), following [5]. This keeps the nearest-neighbor distance from varying with the number of APs. APs are confined to two rows of the lattice with the bottom row having $\max(2, \lceil \frac{N_{AP}}{2} \rceil)$ APs.

Each AP then draws a channel uniformly at random from channels 36, 40, 44, and 48. Random assignment biases the data toward more in-channel contention and interference which could potentially make the problem harder than if we plan out the channels to be minimally interfering. However, TGax sets precedent for random assignment across four channels in its

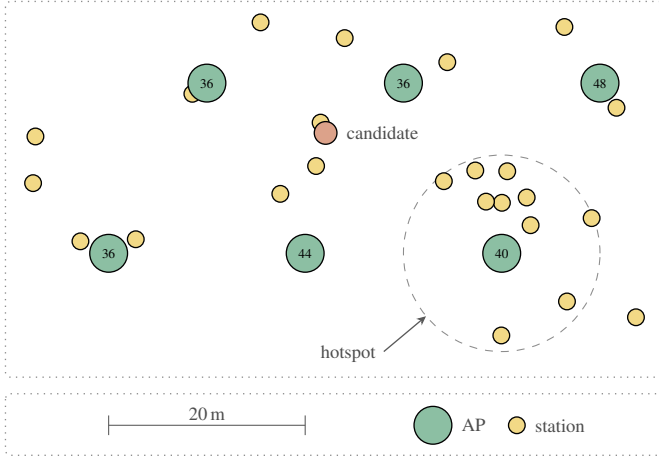


Fig. 1. An example simulated deployment.

enterprise scenario, and across the non-overlapping set in its residential one [5], so this is the strategy adopted.

C. Count, Topology of background STAs

The total number of stations is decided by independently drawing a count of 3, 4, or 5 N_{AP} times and summing them.

A quarter of scenarios spread the stations uniformly. The rest declare one or more APs as “hotspots” around which stations are clustered, keeping the total station count fixed [6]. Such mixing of uniform and deliberately clustered regimes follows common practice in WLAN studies [3], [4], and is of particular import here because under uniform load every AP is about equally busy and the problem we examine collapses to signal strength in a fashion uncharacteristic of reality.

We draw the number of hotspots at random with fewer hotspots being more likely and the count capped at $\lceil N_{AP}/3 \rceil$. The area around a hotspot AP, a 10 m disc around it, is assigned a probability density 4 times that of non-hotspot areas. Each station’s position is drawn from this distribution.

D. Association and Load of background STAs

Each station associates with the AP whose received power is highest. This power is calculated with a log-distance term, then perturbed by a shadowing term: a random sample from a Gaussian of 5 dB standard deviation. Without the perturbation, stations form exact Voronoi partitions, which no real deployment does; 5 dB is the indoor shadowing deviation TGax specifies [5].

Every station then draws its own offered load, independently, from a log-normal distribution with median 2.25 Mbit/s and shape 0.9, capped at 25 Mbit/s, for a mean of 3.37 Mbit/s. Traffic volumes are log-normal rather than Gaussian [7], and the right skew is what puts most stations on a light load and a few on a heavy one without needing a mixture. Its quartiles, about 1.2 and 4.1 Mbit/s, cover browsing and standard-definition streaming, and the upper tail reaches high-definition video.

E. Placement of candidate STA

The candidate station is placed uniformly at random over the same area as the background stations: the AP footprint plus a 10 m border.

F. Data Capture

Background traffic starts at $t = 0.5$ s. Each station’s arrivals are Poisson at its own data rate, and its first packet is delayed by a phase drawn uniformly over one mean interval.

Passive recording runs from $t = 2$ s to $t = 8$ s, covering 6 s of loaded traffic. It starts 1.5 s after the traffic because Minstrel-HT needs about that long to converge, and until it does every scenario carries the same opening burst of retransmissions.

The candidate starts joining at $t = 8$ s, the moment recording ends. After joining, the candidate offers 100 Mbit/s in 1250-byte packets until the run ends at $t = 20$ s. That rate keeps it backlogged, so its measured throughput reflects the service the network gives it rather than the limit of its own traffic generator. We define the throughput label as the mean rate delivered to the candidate from the moment of association to $t = 20$ s; a candidate that never associates scores 0.

G. Matched Counterfactual Replays

For each sampled topological arrangement of the candidate + environment, the experiment runs once per target AP. AP and station positions, background traffic, the candidate’s position, and every pre-association device stay fixed; only the AP selected at $t = 8$ s changes. A `topologySeed` fixes the deployment: station placement, channel selection, and the load offered by each station. Independently, `candidateSeed` fixes the candidate STA’s position, and `rngSeed` controls all other randomness such as fading, contention, rate adaptation, and arrival times of background packets.

Table I summarises the deployment parameters and sampling distributions.

TABLE I
SIMULATOR PARAMETERS AND SAMPLING DISTRIBUTIONS

Quantity	Distribution
AP count	uniform on $\{2, 3, 4, 6, 8\}$
AP channel	uniform on $\{36, 40, 44, 48\}$, iid per AP
Station count	sum over APs of uniform on $\{3, 4, 5\}$
Spatial regime	uniform with prob. 1/4, else hotspots
Hotspot count	$1 \dots \lceil N_{AP}/3 \rceil$, fewer more likely
Hotspot field	$4 \times$ station density inside a 10 m disc
Station load	log-normal, median 2.25 Mbit/s, shape 0.9, cap 25
Association	strongest received power + $\mathcal{N}(0, 5^2)$ dB
Candidate position	uniform over the station area

III. DATASET CONSTRUCTION

A. Recording and Projection

A deployed client scans across all available channels in a rotating fashion. The simulation records more than any deployed client can be expected to observe by recording activity on every occupied channel simultaneously. We then project

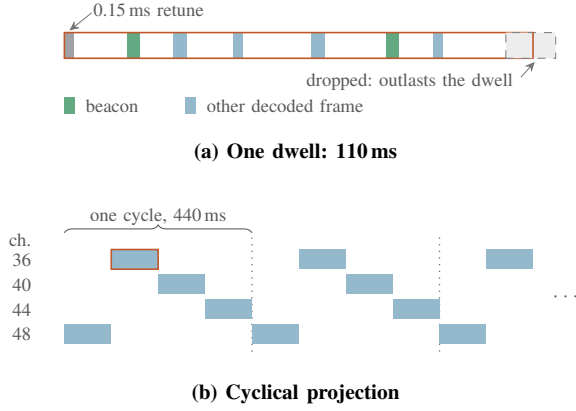


Fig. 2. The all-channel recording projected onto one passive radio.

this complete record down to what a single scanning radio would see (Fig. 2).

Specifically, the candidate STA carries one receive-only scanner PHY per occupied channel, plus a separate PHY reserved for association. The scanner PHYs never transmit and never associate; the association PHY is parked on unused channel 149 until the decision time. At $t = 8$ s the scanner PHYs move to the parking channel, and the association PHY moves into the channel of the AP it intends to associate with.

Two traces are recorded. The monitor-sniffer trace holds every frame a scanner decodes, with the start and end of the transmission carrying it, frequency, BSSID, transmitter, frame category, beacon and retry flags, length, RSSI, airtime, and PHY rate. Frames aggregated into one transmission share their start and end timestamps.

The PHY-state trace gives the fraction of each 1 ms interval in which carrier sense marked the channel busy. This clear-channel-assessment (CCA) measurement registers energy a frame-only view misses: interference, and transmissions too distant to decode. Both thresholds are left at the ns-3 defaults: -62 dBm of received energy from any source, or -82 dBm from a signal whose preamble the PHY recognizes. Most driver stacks expose an equivalent channel-busy counter, though availability varies by platform.

The projection onto one passive radio is done as follows: One radio is assumed to rotate through the available channels in a certain randomly decided order, dwelling 110 ms on each with 0.15 ms to retune, and repeats the cycle for as much of the observation window as possible. Only an integer number of all-channel cycles are retained in the dataset, and the last few dwells may be excluded to this effect. A frame is retained in the dataset only if its whole transmission falls inside a dwell after retuning; CCA intervals outside a dwell are discarded as well. A 110 ms dwell spans just over one 102.4 ms beacon interval, so each visit yields about one beacon from every AP on that channel. Finally, the RSSI of each frame is quantized to 1 dB when creating the dataset so as to match practical precision.

B. Training Data

The corpus used in the making of this report simulated 1,000 deployments, placing the candidate at five positions in each of them. Seven of the resulting 5,000 “scans” heard beacons from fewer than two APs and were dropped, having no choice to offer.

Table II summarises the resulting corpus.

TABLE II
THE CORPUS, AFTER PROJECTION ONTO ONE RADIO.

Deployments	1,000
Scans, of 5,000 drawn	4,993
Simulated runs	22,925
Discovered APs per scan	2 to 8
Scans hearing every AP placed	77.4%
Throughput label	0 to 59.0 Mbit/s
median, mean	5.3, 11.3 Mbit/s
exactly zero	14.0% of rows

The label is right-skewed, its mean at twice its median, and the zero rows are due to APs that were heard but could not usefully be joined by a far away candidate.

Training splits of this dataset were drawn by topology rather than by scan. The five scans of a deployment differ only in where the candidate stands: the APs, their channels, the background stations and the traffic they offer are identical across them, so testing on a scan whose deployment also appears in training would score the model on something close to what it was fitted on. Deployments are dealt 60% to training, 20% to validation and 20% to test, each AP count dealt separately so that no split ends up with a disproportionate share of the largest or smallest deployments.

IV. MODEL ARCHITECTURE

The model adapts a pretrained language model to this problem following the structure of NetLLM [9]: A learned encoder turns the observation into a sequence of vectors, a frozen decoder-only backbone reads that sequence as input embeddings, and only low-rank adapters [10] on its attention projections are trained.

Every model in this section scores the *discovered APs* of a scan: those whose beacons it decoded at least once. An AP never heard leaves nothing in the observation to read from, so the arg max of Section I is in practice taken over the discovered APs rather than over every AP in the deployment. This is a limit of passive observation rather than of the models, and it costs little, since an AP too distant for its beacons to arrive is unlikely to be the one that would have served best.

A. Embedding

The observation window contains several thousand frames and carrier sense datapoints, making attending over the scan frame by frame impractical. Further, several meaningful signals lie as aggregates or counts which would not be best captured by such a model.

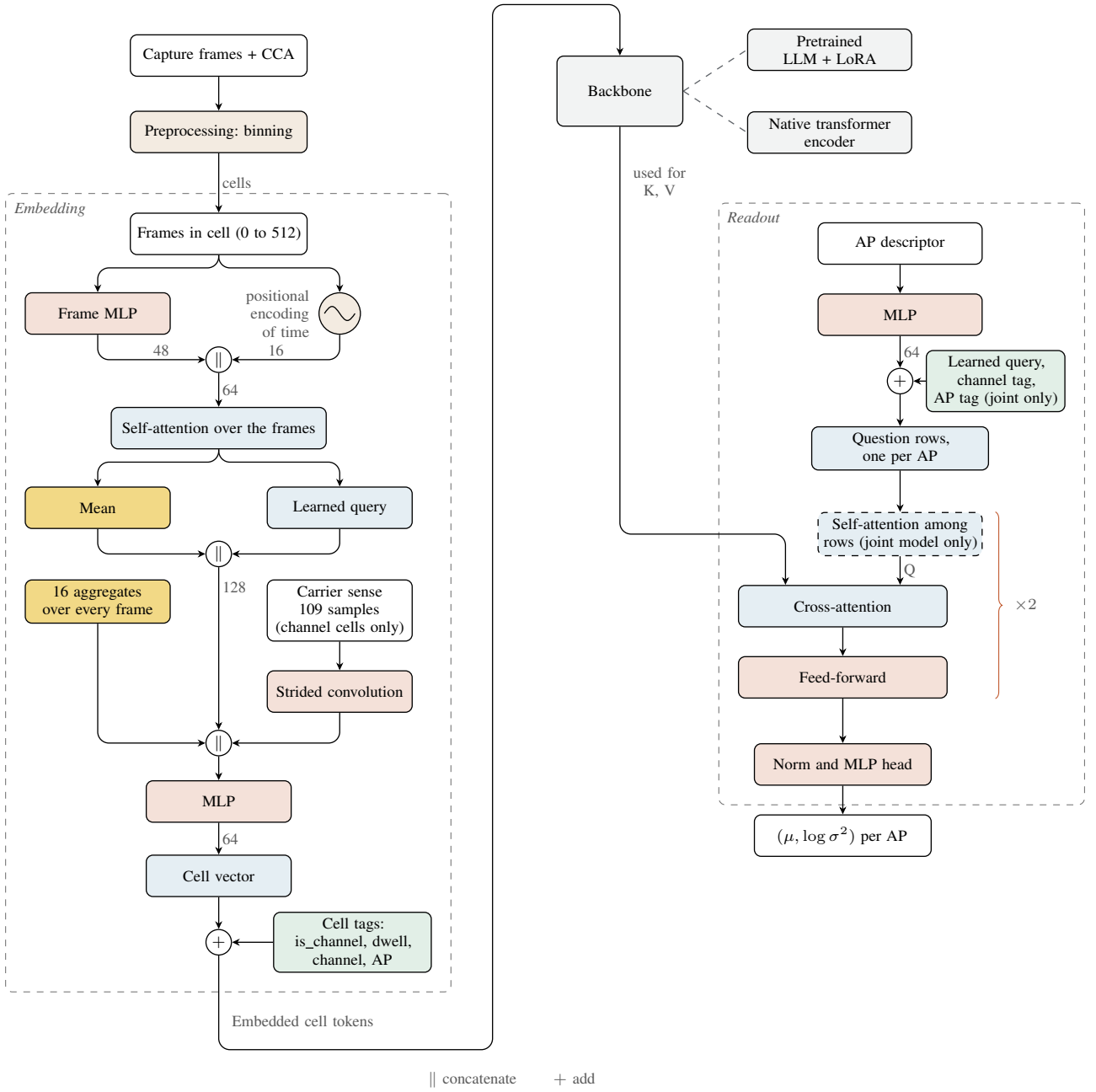


Fig. 3. Model architecture.

Thus, we group the frames of a scan into *cells*, each of which the model reduces to a single 64-dimensional vector. First, the scan is cut into bins over time. Each bin is chosen to be of 110 ms, such that it coincides with channel dwells, keeping same-channel frames together. For each bin, we emit one channel cell with all the frames of the dwell, and one cell per discovered AP on that dwell's channel, containing only the frames that carry its BSSID. A six-second scan of 13 cycles thus yields, in each cycle, four channel cells and one cell per discovered AP.

The model then learns a representation for each cell by allowing frames within the cell to attend amongst themselves.

A maximum of 512 frames are used for this purpose to limit quadratic attention costs contributed by a few outlier cells.

Frames carry 10 numerical values and a categorical flag; 9 numbers and the category are embedded into a 48-dimensional space with a 2 layer MLP. Non-linearity is needed here to capture relations between the frame characteristics, some of which are boolean, for attention to better relate frames to each other. The tenth value, time, is positionally encoded. The time from the start of the dwell is encoded as 8 sine-cosine pairs, with periods falling geometrically from 220 ms to 0.2 ms [11]. The fast end is about the airtime of one frame, below which there is nothing to distinguish; the slow end is twice the longest

gap a cell can hold, so that the sine-cosine pair turns less than half a circle over any gap within a dwell and orders it unambiguously. The 8 pairs are appended to the aforementioned embedding, creating an overall embedding dimension of 64 for each frame.

The original transformer *adds* the positional encoding to the token embedding, rather than concatenate, so the position and the content share all coordinates [11]. This has a notable advantage that both content and position are able to use the full dimensionality of the embedding, allowing for greater expressibility of both of them. The drawback with this approach is that content and position are mixed into the same values and separating out that information is left to training, and is typically achieved only with large volumes of training data.

When the inputs are timestamped observations rather than ordered tokens, however, there is actually considerable precedent for the encoding to be concatenated [14], [15]. Nagesh *et al.* [16] compare the two arrangements on sequences of timestamped records and find concatenation ahead in every setting they test, by a margin that narrows as they add training data. With a corpus of a few thousand scans, we opt to concatenate.

The attended frames must now be reduced to a single vector that represents the entire cell. To achieve this, we use one learned query attending over them, the pooling of the Set Transformer [18], and in addition, we concatenate a mean of all attended values. The mean reports the cell as a whole, while the query is free to weight a few frames above the rest.

Two exact summaries join the pooled vector: the first is 16 aggregates taken over every frame the cell held, including those the cap of 512 dropped. Table IV lists the quantities that are aggregated here. Such counts and totals cannot be reported by the pooled set of vectors and are thus directly given to the model. The second summary belongs to channel cells alone: the 109 carrier-sense samples of the dwell, one per millisecond, read by a strided one-dimensional convolution. (One CCA sample is lost to radio retune.)

A final MLP maps the concatenation of the summaries and the pooled vector to a single 64 dimensional vector leaving the encoder (Figure 3). Both an AP and a channel cell share the frame embedding and the self-attention, but each keeps its own pooling query and its own output MLP, since only channel cells carry CCA signals, and an aggregate such as beacon level does not mean the same thing on the two and should not be afforded the same mapping. A cell that heard nothing has nothing to pool and takes a learned vector in place of both reductions.

The resulting cell vector, although containing a summary of all its frames, mostly lacks identifiers to distinguish itself or relate to other cells. We thus add tags for the channel it was heard on, whether it is an AP or channel cell, and a positional embedding for the dwell number it belongs to. Each tag is a 64 dimensional vector added onto the cell vector, its direction drawn once from a random orthogonal set and never trained. Tags from different sets are orthogonal, as are the tags within the cell type and channel sets. The dwell tags are

instead sinusoidal in the dwell index, so that dwells close in time take similar tags. All tags have a trainable length shared within their kind, initially 0.5. Finally the AP’s identity must be added to the representation of AP cells, for which we take two approaches:

- Declare one AP as the target AP whose throughput we are interested in. We now code the identity relative to this AP: `TARGET_AP` for the target’s own cells, `CO_CHANNEL_AP` for an AP sharing its channel, and `OTHER_AP` for the rest. These three roles use learned vectors. Inference runs once per target AP, with a different set of roles added to the cell vectors each time.
- Assign each AP in the scan a numerical identity and look up a fixed tag direction. The direction is shared by that AP’s cells and its readout row; its length is learned. Identities are randomly relabelled for each training sample, so the model cannot attach a persistent meaning to an AP number. This lookup fixes the maximum number of APs handled at once, but permits all their throughputs to be predicted from one pass through the backbone.

B. Backbone

The tagged cell vectors attend across the scan so that a cell describing one AP can incorporate the activity of other APs and channels. We consider two backbones for this step.

The custom backbone is a transformer encoder trained from scratch, with two blocks of four-head self-attention at width 64. Each block normalises its input before attention, adds the attended values to the input, then applies a feed-forward MLP with another normalisation and residual connection; that MLP widens to 256 and back. Each block also adds to its attention logits a learned scalar per head for the channel distance between the two cells, in 20 MHz steps. Every cell can attend to every observed cell; dwell order is supplied by the tags.

The second follows the adaptation of a pretrained language model described above. Linear projections map the cell vectors from 64 to 2048 dimensions, then back after Llama-3.2-1B [12] has processed them. The backbone weights remain frozen, with LoRA adapters of rank 8 and $\alpha = 32$ on the query and value projections, at dropout 0.05. Its attention mask is block causal, permitting cells to attend within their own dwell and to earlier dwells. Cells in the same dwell receive the same rotary position, since their order within that dwell carries no meaning [13].

The target-AP model runs the backbone separately for each AP, because changing the target changes the role tags on its input. The joint-AP model runs it once for the scan.

C. Decoder

The backbone returns one vector per cell, while we need one prediction per AP. To obtain it, we construct a single learnable query per AP. (In the target model, this is just one AP.)

We provide additional context to the query, by means of an embedded representation of what could identify the AP.

We take the AP’s channel tag, the AP’s identity tag (applicable only to the joint model), and an AP descriptor: a whole-window summary of its beacons and its BSS (Exact

quantities listed in Table IV.) and project it to 64 dimensions with a two-layer MLP.

The queries thus created pass through two decoder blocks. Each block lets the queries attend to the backbone’s cell vectors, using the rows as queries and the cells as keys and values. The block ends with a standard feed-forward MLP. Both operations have normalisation and residual connections. In the joint model, self-attention between AP query rows precedes cross-attention, allowing each prediction to use what the other rows have learned. The target model has only one query row so it omits this step.

In cross-attention, we add two learned biases to the query–key score before softmax. The first depends on the separation between the query AP’s channel and the cell’s channel; the second applies when an AP cell belongs to the AP being scored. Each head learns its own biases, shared across the two decoder blocks. These adjust the attention weights while leaving every observed cell available.

A final MLP maps each row to a mean and log variance of standardised $\log(1 + y_i)$, where y_i is throughput in Mbit/s. Training begins with squared error on the mean and then uses Gaussian negative log likelihood to fit both outputs. The predicted means order the APs for selection.

V. ARCHITECTURE CHOICES

A. Frame Time and Cell Identity

We concatenate time embeddings with frame content, but *add* dwell embedding to cell content following common practice. Concatenation is motivated through the separation of content and time, but this argument actually needs further qualification.

The output of attention layers can write context into the timestamp coordinates as well as the content coordinates, so even though residual addition retains the original input, the fields end up becoming a combined representation nonetheless in subsequent layers.

To avoid this problem, TUPE [17] maintains separation in the attention scores themselves. It computes content–content and position–position scores with separate projections, then sums them before a single softmax. The positional encoding remains separate from the contextual content across layers. Concatenation alone does not impose this structure: its query and key projections can produce mixed content–time scores even in the first layer.

The empirical precedent survives this qualification. Nagesh *et al.* [16] concatenate time with observations in a standard six-layer encoder, without TUPE’s separate positional scores, and still find concatenation ahead of addition. Our frame encoder has only one self-attention layer before pooling, so the argument that subsequent layers have greater mixing of fields doesn’t really apply.

At the cell level, the temporal coordinate is a dwell number, advancing at fixed 110ms intervals. Frame timestamps instead describe observations that can cluster in busy periods and become sparse in quiet ones, and thus can be treated as true timestamps whereas dwell numbers are closer to a numbered position. The precedent established by Nagesh *et al.* therefore

applies more closely to frames. There is also a difference in what the content must represent: one frame vector just describes one frame of 10 headers, while a cell vector summarises all of its frames and their aggregates. This means that dedicating dimensions to positional embedding comes at a great cost at the cell level, which could better be used for representational power. Thus, the arguments that support concatenation at the frame level, do not apply strongly at the cell level, and addition remains the norm.

B. Identity in the Attention Logits

Alongside the channel tags, we give channel separation an independent contribution to the attention score. For one head, the logit between query i and key j becomes

$$\ell_{ij} = \frac{q_i^\top k_j}{\sqrt{d_h}} + b(\Delta_{ij}), \quad (2)$$

where d_h is the query and key dimension, Δ_{ij} is the separation between their channels, and b assigns a learned scalar to each possible separation. Thus, pairs with the same channel separation share a bias, regardless of their content. Each head learns its own biases.

In encoder self-attention, Δ_{ij} separates the channels of the query cell and the key cell; in decoder cross-attention, those of the AP being scored and the key cell.

T5 [19] and Swin [20] use learned relative-position biases in attention, while Graphormer [21] uses biases indexed by shortest-path distance between nodes. These provide precedent for adjusting a query–key score using a relationship between the two inputs. However, in our case, channel and AP relationships are also available through the tags, which raises the question of whether a direct bias supplies anything attention could not already learn.

To see the overlap, write an input as content plus tag, $x_i = c_i + t_i$. Let W_Q and W_K be one head’s query and key maps. Temporarily omitting normalisation, projection biases and the attention scale, set $M = W_Q^\top W_K$. Then

$$\begin{aligned} q_i^\top k_j &= c_i^\top M c_j + c_i^\top M t_j \\ &\quad + t_i^\top M c_j + t_i^\top M t_j. \end{aligned} \quad (3)$$

The last term can express preferences for pairs of tags, including matching channels or AP identities. In cross-attention the same expansion applies with AP row content and tags on the query side and cell content and tags on the key side.

However, M is shared by all four terms. Adjusting it to obtain a particular tag preference can also change how content is compared and how content and tags interact. The tag score is therefore tied to the rest of the attention calculation, even when the tag directions themselves are learned. A direct bias gives each relationship its own parameter, adjustable without changing the query and key projections, and shares that parameter across every pair with the same relationship. Our blocks also normalise content and tags together, so the expansion above illustrates the overlap rather than an exact decomposition of their logits. We retain the channel-distance bias in the encoder and both the channel-distance and

TABLE III
1,000-TOPOLOGY CORPUS RESULTS. MAE AND MEAN REGRET ARE IN MBIT/S.

Model	MAE ↓	Top-1 ↑	Mean regret ↓	Spearman ↑
<i>Reference rules: five folds</i>				
Random choice	—	0.287 ± 0.001	14.426 ± 0.590	—
Strongest RSSI	—	0.621 ± 0.025	5.263 ± 0.676	0.615 ± 0.022
Least busy channel	—	0.525 ± 0.033	6.431 ± 0.329	0.391 ± 0.042
RSSI minus busy	—	0.803 ± 0.020	1.386 ± 0.138	0.754 ± 0.024
<i>Tabular predictors: five folds</i>				
Ridge regression	4.892 ± 0.360	0.827 ± 0.016	1.031 ± 0.070	0.803 ± 0.008
Random forest	2.396 ± 0.030	0.878 ± 0.014	0.508 ± 0.058	0.865 ± 0.007
Gradient boosting	2.264 ± 0.082	0.883 ± 0.015	0.448 ± 0.078	0.866 ± 0.007
<i>Custom backbone: five folds</i>				
Target AP	2.452 ± 0.414	0.887 ± 0.020	0.444 ± 0.093	0.879 ± 0.016
Joint AP	2.032 ± 0.067	0.888 ± 0.009	0.422 ± 0.040	0.879 ± 0.015
<i>LoRA fine-tuned backbone</i>				
Target AP	2.038	0.886	0.462	0.885
Joint AP	2.163	0.885	0.485	0.880

Five-fold entries show mean $\pm$ standard deviation. LoRA scores are provisional fold-0 values pending replacement with five-fold results.

own-AP biases in the decoder to give these relationships an independently adjustable contribution alongside the tags.

VI. RESULTS

Table III compares the transformers with passive-selection heuristics and predictors that read whole-window summaries. We use five-fold cross-validation by topology: each rotation uses three folds for training, one for validation and one for testing. Over the five rotations, every topology is tested exactly once, with all its scans kept together.

We report throughput error (MAE), the fraction of scans selecting the best AP (Top-1), throughput lost by the selection (mean regret), and correlation of the predicted and true AP orderings (Spearman). MAE is omitted for the rules.

A. Baselines

We evaluate four reference rules over the discovered APs. *Random choice* selects uniformly. *Strongest RSSI* selects the AP with the highest mean beacon RSSI. *Least busy channel* selects an AP whose channel has the lowest CCA busy fraction, averaged over the dwells on that channel. Tied choices share selection credit and regret uniformly.

RSSI minus busy combines these measurements, scoring AP i as $r_i - kb_i$, where r_i is its mean beacon RSSI in dBm and b_i is its channel's CCA busy fraction. We fit the single coefficient k on training scans, choosing from $\{0, 5, \dots, 80\}$ the value with the lowest mean regret. At $k = 0$ this reduces to strongest-RSSI selection.

The tabular predictors read 32 whole-window features per discovered AP: 10 summarise its channel, 16 its own BSS, and 6 its relationship to the other discovered APs. These include CCA occupancy, frame and byte rates, retries, beacon RSSI and spacing, observed transmitter counts, and relative signal strength and airtime shares. They provide a comparison with both linear and nonlinear fits of summary statistics.

Ridge regression fits a linear predictor after standardising the features on training data, with an L_2 penalty selected from

$\alpha \in \{0.1, 1, 10, 100\}$. *Random forest* averages 400 regression trees; we vary the minimum leaf size over $\{1, 2, 4\}$ and the number of features considered at each split between the square root of the feature count, half the features, and all features. *Histogram gradient boosting* fits trees sequentially to correct prediction errors. We try 200 or 400 iterations, learning rates in $\{0.03, 0.06, 0.1\}$, and minimum leaf sizes in $\{4, 8, 16\}$, with L_2 regularisation fixed at 1.

Each tabular configuration is fitted against either throughput or $\log(1 + y_i)$. Validation mean regret selects the configuration and target transform within each family, with Spearman correlation breaking ties. The test topologies are held out from both fitting and selection.

B. Transformer Training

The custom backbone of Section IV-B contains 0.100 million parameters across its two blocks. The target-AP and joint-AP models built on it train 0.313 and 0.347 million parameters, including the embedding, decoder and readout layers on top of the backbone.

In contrast, the LoRA backbone keeps the 973 million parameters of Llama-3.2-1B frozen, after removing its unused vocabulary embedding, and trains 1.116 million: 0.852 million in its rank-8 adapters and 0.264 million in the width projections. With the same embedding, decoder and readout layers, the target-AP and joint-AP models built on it train 1.330 and 1.363 million parameters.

We train the transformer models for 60 epochs with AdamW and cosine learning-rate decay. The models with the custom backbone use an initial learning rate of 10^{-3} and weight decay of 10^{-2} ; the LLM variants use 10^{-4} for both. Throughput predictors minimise Gaussian negative log likelihood on standardised $\log(1 + y_i)$, averaged over the discovered APs in each batch. The first five epochs fit the mean with squared error before introducing the predicted variance. Validation regret

selects the checkpoint, with Spearman correlation breaking ties.

C. Comparison

Signal strength alone picks the best AP in 62.1% of scans and gives up 5.26 Mbit/s on average (Table III). Channel occupancy alone does worse, at 52.5% and 6.43 Mbit/s. Combined under one fitted weight, the two reach 80.3% and a regret of 1.39 Mbit/s, about a quarter of the strongest-RSSI figure.

Every learned predictor improves on the rules. Ridge regression reaches a regret of 1.03 Mbit/s. The random forest, gradient boosting and both transformers fall between 0.42 and 0.51 Mbit/s, with Top-1 rates from 0.878 to 0.888. The joint-AP model has the lowest mean regret, 0.422 Mbit/s, against 0.444 for the target-AP model and 0.448 for gradient boosting. These gaps are smaller than the spread across folds. Compared fold by fold, the joint-AP model has lower regret than gradient boosting on three of the five folds, and the target-AP model on four.

Throughput error does not track selection. The joint-AP model has the lowest MAE, 2.03 Mbit/s. The target-AP model's MAE varies most across folds, and its fold with the highest MAE, 3.07 Mbit/s, is also its fold with the lowest regret, 0.289 Mbit/s.

The LoRA backbone has so far been evaluated on fold 0 alone. On that fold the custom backbone's target-AP and joint-AP models reach regrets of 0.465 and 0.435 Mbit/s; with the LoRA backbone they reach 0.462 and 0.485 Mbit/s. Top-1 rates move from 0.891 and 0.890 to 0.886 and 0.885. The pretrained backbone thus gives comparable selection on this fold, while training about four times as many parameters alongside 973 million frozen ones.

D. Discussion

NetLLM [9] reports adapted language models outperforming task-specific networks on each of its networking tasks. Our fold-0 comparison sits closer to Tan *et al.* [22], who find that removing the language model from LLM-based time-series forecasters, or replacing it with a single attention layer, does not degrade forecasting. Here a two-block encoder trained from scratch matches the frozen 1B-parameter backbone.

The LoRA backbone retains two properties the scores do not measure. Its adapters and task layers can be refitted to another task, or to captures from real deployments, while the backbone itself is shared, although the custom models can be fine-tuned as well and transfer to real captures is untested. Its block-causal mask also lets it encode a prefix of a scan without seeing later dwells. The custom backbone accepts fewer cells too, but each cell's representation changes as later cells arrive. A shorter window also changes which APs are discovered and what their descriptors hold. Neither backbone has been evaluated on prefixes, so selection from, for instance, one second of scanning remains open.

Gradient boosting's closeness to the transformers may follow from how traffic is simulated. Arrivals and contention fluctuate within a scan, but every station keeps its mean load and none

starts or stops during the observation, so summaries over the whole window may hold most of what selection needs. Loads that change within the scan would test what the cell sequence adds over those summaries.

VII. FURTHER STUDY

A. AP-Advertised BSS Load

B. Traffic That Changes During the Scan

Bursty sessions. Random Changes in offered rate. Stations randomly starting or stopping?

C. Shorter Observation Windows

D. Adding realism to the Simulator

E. Testing Beyond the Simulated Deployment

REFERENCES

- [1] P. Dely, A. Kessler, L. Chow, N. Bambos, N. Bayer, H. Einsiedler, and C. Peylo, "BEST-AP: Non-intrusive estimation of available bandwidth and its application for dynamic access point selection," *Computer Communications*, vol. 39, pp. 78–91, 2014, doi: 10.1016/j.comcom.2013.11.006.
- [2] T. Adame, M. Carrascosa, B. Bellalta, I. Pretel, and I. Etxebarria, "Channel load aware AP/Extender selection in home WiFi networks using IEEE 802.11k/v," *IEEE Access*, vol. 9, pp. 30095–30112, 2021, doi: 10.1109/ACCESS.2021.3059473.
- [3] W. Wong and S.-H. G. Chan, "Distributed joint AP grouping and user association for MU-MIMO networks," in *Proc. IEEE INFOCOM*, 2018, pp. 252–260, doi: 10.1109/INFOCOM.2018.8486284.
- [4] Y. Zhang and L. Dai, "Joint optimization of placement and coverage of access points for IEEE 802.11 networks," in *Proc. IEEE ICC*, 2020, pp. 1–7, doi: 10.1109/ICC40277.2020.9149098.
- [5] IEEE 802.11 Task Group ax, "TGax simulation scenarios," IEEE 802.11-14/0980r16, 2016. [Online]. Available: <https://mentor.ieee.org/802.11/dcn/14/11-14-0980-16-00ax-simulation-scenarios.docx>
- [6] 3GPP, "Evolved Universal Terrestrial Radio Access (E-UTRA); Further advancements for E-UTRA physical layer aspects," TR 36.814 V9.0.0, Mar. 2010.
- [7] M. Alasmar, R. Clegg, N. Zakhleniuk, and G. Parisi, "Internet traffic volumes are not Gaussian — they are log-normal: An 18-year longitudinal study with implications for modelling and prediction," arXiv:2007.10150, 2020.
- [8] ns-3 Project, *ns-3 Model Library, Release 3.45*, 2025. [Online]. Available: <https://www.nsnam.org/docs/release/3.45/models/>
- [9] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proc. ACM SIGCOMM*, 2024, pp. 661–678, doi: 10.1145/3651890.3672268.
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [12] Meta, "Llama 3.2-1B," model card, 2024. [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.2-1B>
- [13] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, art. 127063, 2024, doi: 10.1016/j.neucom.2023.127063.
- [14] D. Xu, C. Ruan, S. Kumar, E. Korpeoglu, and K. Achan, "Self-attention with functional time representation learning," in *Proc. NeurIPS*, 2019, pp. 15915–15925.
- [15] M. Horn, M. Moor, C. Bock, B. Rieck, and K. Borgwardt, "Set functions for time series," in *Proc. ICML*, 2020, pp. 4353–4363.
- [16] S. Nagesh *et al.*, "Transformers for prompt-level EMA non-response prediction," arXiv:2111.01193, 2021.
- [17] G. Ke, D. He, and T.-Y. Liu, "Rethinking positional encoding in language pre-training," in *Proc. ICLR*, 2021.
- [18] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, "Set Transformer: A framework for attention-based permutation-invariant neural networks," in *Proc. ICML*, 2019, pp. 3744–3753.

- [19] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [20] Z. Liu *et al.*, “Swin Transformer: Hierarchical vision transformer using shifted windows,” in *Proc. IEEE/CVF ICCV*, 2021, pp. 10012–10022.
- [21] C. Ying *et al.*, “Do transformers really perform bad for graph representation?” in *Advances in Neural Information Processing Systems*, vol. 34, pp. 28877–28888, 2021.
- [22] M. Tan, M. A. Merrill, V. Gupta, T. Althoff, and T. Hartvigsen, “Are language models actually useful for time series forecasting?” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: <https://arxiv.org/abs/2406.16964>

APPENDIX A MODEL INPUTS

Table IV lists every quantity the models read, in the three groups of Section IV: frame features enter the frame embedding, cell aggregates join each pooled cell vector, and AP descriptors build the decoder’s query for each discovered AP. Fractions and rates are computed over the time the radio actually listened on the relevant channel, which is about a quarter of the observation window. Counts and durations marked $\log_{1p}$ are stored as $\log(1 + x)$.

TABLE IV
THE MODEL INPUTS. FRAME FEATURES ARE READ ONCE PER DECODED FRAME, AGGREGATES ONCE PER CELL OVER EVERY FRAME IT HELD, AND DESCRIPTORS ONCE PER DISCOVERED AP OVER THE WHOLE WINDOW.

Column	Meaning
Frame	
offset_in_dwell	Where in the 110 ms dwell the transmission began, 0 to 1; encoded as sine–cosine pairs rather than embedded
gap_loglp	Idle time on this channel before it
rss_i_dbm	Received level
airtime_loglp	The frame’s own airtime, not that of the transmission carrying it
length_loglp	Length in bytes
rate_loglp	PHY rate
is_beacon, is_retry	Beacon and retry flags
from_ap	The BSSID itself sent it, not one of its stations
of_discovered_ap	Carries a discovered AP’s BSSID
category	Management, control or data; embedded rather than numeric
Cell aggregate	
frames_loglp, bytes_loglp	Frames and bytes the cell held
airtime_fraction	Their own airtimes summed, over the time listened
busy_fraction	Time with a decoded transmission on air, overlaps counted once
data_fraction, mgmt_fraction	Share of frames by category
retry_fraction	Share marked retry
transmitters_loglp	Distinct transmitters; on an AP cell its own is left out
beacons_loglp	The AP’s beacons; 0 on a channel cell
beacon_rssi_mean	Their level; missing on a channel cell
rss_i_mean, rss_i_max	Over every frame, its stations’ as well as the AP’s
data_rate_loglp	Mean PHY rate of the data frames
longest_gap_fraction	Longest stretch with nothing decoded, over the time listened
cca_mean, cca_max	Carrier sense over the dwell; 0 on an AP cell
AP descriptor	
beacon_rssi_{mean, std, min, max, last}	Beacon level over the whole window: mean, spread, extremes, and most recent
beacons_loglp	Beacons decoded
bss_airtime_fraction	Its BSS’s summed airtime, over the time listened on its channel
bss_transmitters_loglp	Distinct stations heard sending in its BSS
cochannel_discovered_aps	Other discovered APs on its channel
n_discovered_aps	Discovered APs in the scan